Experiment 11: CNN on CIFAR-10 Subset (0–3 Classes)

Theory:

Convolutional Neural Networks (CNNs) are specialized deep learning models particularly effective for image classification tasks. In this experiment, a CNN is trained on a subset of the CIFAR-10 dataset, limited to the first four classes: airplane, automobile, bird, and cat. The CNN learns hierarchical features from raw pixels using convolutional layers, followed by pooling and dense layers.

Algorithm:

1. Load the CIFAR-10 dataset and extract only classes 0 to 3.
2. Normalize pixel values to the range [0, 1].
3. One-hot encode the labels for multiclass classification.
4. Build a CNN with:
   - Two convolutional + max-pooling layers.
   - One dense layer with dropout.
   - Final softmax layer with 4 outputs.
5. Compile using Adam optimizer and categorical crossentropy loss.
6. Train the model and plot accuracy/loss across epochs.

Flowchart (Written Format):

Start → Load CIFAR-10 → Filter Classes 0–3 → Normalize Pixels → One-Hot Encode Labels → Build CNN → Compile Model → Train CNN → Evaluate → Plot Accuracy/Loss → End

Python Code:

```
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from keras.datasets import cifar10
from keras.utils import to_categorical
import numpy as np
import matplotlib.pyplot as plt

(X_train, y_train), (X_test, y_test) = cifar10.load_data()
y_train = y_train.flatten(); y_test = y_test.flatten()

selected_classes = [0, 1, 2, 3]
train_filter = np.isin(y_train, selected_classes)
test_filter = np.isin(y_test, selected_classes)
X_train, y_train = X_train[train_filter], y_train[train_filter]
X_test, y_test = X_test[test_filter], y_test[test_filter]

X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0

class_map = {c: i for i, c in enumerate(selected_classes)}
y_train = np.vectorize(class_map.get)(y_train)
y_test = np.vectorize(class_map.get)(y_test)
y_train = to_categorical(y_train, num_classes=4)
y_test = to_categorical(y_test, num_classes=4)

model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(64, activation='relu'),
```

```
    Dropout(0.5),
    Dense(4, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
history = model.fit(X_train, y_train, epochs=10, batch_size=64, validation_data=(X_test, y_test))
```